Filter Feature Regression

Workflow Validation

2026

FILTER FEATURE REGRESSION AND WORKFLOW DEMONSTRATION

Purpose

This is a self-contained .md fixture for the Markdown-to-PDF workflow. Compile this file by itself with compile.bat. A successful build exercises the real Windows wrapper, Pandoc, config\markdown-normalizer.lua, config\code-fence-runner.lua, config\custom.tex, and XeLaTeX. Most Python examples use the standard library. The probability graph examples require NumPy and Matplotlib and skip cleanly if either package is unavailable.

This is a broad, repeatable regression sample, not a mathematical proof that every possible input is supported. Expected-failure cases are listed near the end and are intentionally not activated as build-breaking input. This fixture is not intended to be rebuilt after every small code or documentation change. Contributors should choose focused tests for routine changes; use this full fixture when its behavior is affected, for end-to-end validation, or before a release. Any user can run it at any time to check the workflow. Every feature addition or bug fix must still be tested with a focused test input through compile.bat; that targeted test does not require rebuilding this full fixture.

Run this exact fixture

In File Explorer, drag this file onto compile.bat, or run the following from PowerShell after changing to the tool folder (the folder containing compile.bat):

```
.\compile.bat ".\documentation\filter-feature-regression.md"
```

The template and filters are read from the config folder beside compile.bat. The PDF should be created beside this .md file with the same base name. Expected console notices include summaries for intentionally cleaned citation markers / unsupported emoji, Python consent prompts for uncached/forced runs, and the deliberately failing Python demonstration. Review and approve only the fixture's test code; forced fresh-run examples ask on every build. The build should still complete and produce a PDF.

1. Document metadata, order, and Markdown basics

The YAML title and author below exercise YAML metadata in a .md input. The title in the produced PDF should be "Filter Feature Regression".

The first input's content order is preserved. This standalone file is one input. Its multi-file behavior is tested separately below with filter-feature-regression-part-2.md.

Multi-file regression markers from input 1: MULTI_INPUT_PART_ONE_TEXT

Python error - this block did not finish:

```
No installed Python found!
```

Markdown supports **bold**, *italic*, ~~strikeout~~, `inline code`, links, lists, headings, and ordinary paragraphs. This is a link to Pandoc.

Checklist syntax note: `[ ] item` without a list marker is ordinary paragraph text, not a list. Use `- [ ] item` for a Pandoc task-list item (or `- item` for an ordinary bullet). This fixture uses ordinary bullets in its final checklist.

Bare URLs are autolinked by the cleaned-input reader: https://pandoc.org/installing.html

The `<br>` tag outside code becomes a line break.\newline HTML subscript and superscript are converted: H\textsubscript{2}O and x^{2}.

Code-like text is protected from prose rewriting: `<br>`, `\maketitle`, and `\usepackage{fakepackage}` remain literal here.

Raw HTML nodes represented by Pandoc as raw blocks (for example scripts) are removed rather than passed to XeLaTeX:

Pandoc represents an HTML div as a structural Div, not a raw HTML block. Its ordinary text content is retained by this filter:

HTML div content is retained as document text.

The same special-looking text is protected inside a verbatim environment:

\begin{verbatim} \maketitle \usepackage{fakepackage} \end{verbatim}

Lists and inline formatting

- Apple costs $5.
- Pear costs $7.
- A third item stays separate from the currency examples.

1. Ordered item with **bold** text.
2. Ordered item with *emphasis*.
3. Ordered item with inline math, $a^2 + b^2 = c^2$.

   Indented code remains code: \maketitle \usepackage{fakepackage}

Pipe table and math inside a table

Quantity	Value
Inline math	$x^2 + y^2$
Greek	$\alpha + \beta$
Bold around nested complex math	**Bold *emphasis before* $\begin{matrix} a & b \\ c & d \end{matrix}$ and after remains bold**
Pipe-like math outside the table	See the display equation below

The table exercises ordinary pipe-table parsing, alignment, inline math, and Unicode-capable LaTeX output. Complex matrix math is kept outside formatting spans that break in table cells, including when nested in emphasis; surrounding text formatting remains intact.

2. Math and Unicode handling

Inline math: $E = mc^2$, $x_1 + x_2$, and $\alpha \leq \beta$.

Display math:

$$\int_0^1 x^2\,dx = \frac{1}{3}.$$

An aligned raw-LaTeX math environment: \begin{align} (a+b)^2 &= a^2 + 2ab + b^2, \ (a-b)^2 &= a^2 - 2ab + b^2. \end{align}

The next display contains a line that starts and ends with vertical bars. It must remain math, not be reinterpreted as a Markdown table:

$$|a + b| \le |a| + |b|$$

Price-like list items above must not be joined into one formula. A real multi-line single-dollar formula may continue across lines when its content clearly contains math:

$x + y = z$

Unicode examples cover Greek letters, capital Greek / Latin-lookalike letters, operators, arrows, relations, and stacked superscripts/subscripts:

Greek: $\alpha\ \beta\ \gamma\ \Delta\ \Omega\ \Upsilon$. Relations: $\le \ge \ne \approx \in \emptyset \forall \exists$. Arrows: $\to \Rightarrow \leftrightarrow$. Scripts: x^2, x^{-1}, H_2O, and $_{12}$. Template symbol-font examples: ■ □ ● ○ ★. Meaning-bearing status marks: ✓ × ! (from ✓ × !).

Unicode script characters in source-code listings are kept in the code and declared for XeLaTeX when needed. Some glyphs still depend on installed fonts.

3. Real citations and AI-marker cleanup

This citation uses inline CSL reference metadata from the YAML block: (Doe 2024).

The following AI-generated citation artifacts should be removed while their removal count is reported to the console:

An old marker and an old citation are removed. The bracketed source marker is removed.

Emoji with meaning are mapped to printable check/cross/warning marks above. An unsupported emoji in ordinary prose () is removed and counted. Citation cleanup is for known AI artifacts; it is not a replacement for real citations.

4. Pasted LaTeX and template integration

Commands in ordinary pasted LaTeX are moved to the preamble when supported. Template-loaded packages such as siunitx and tcolorbox are not loaded a second time, preventing option clashes. Multiple usepackage commands may share a line: \usepackage{siunitx}\usepackage{tcolorbox}

The template's siunitx/physics quantity dispatcher supports: \qty{1.5}{\meter}

Theorem-like environments are converted to Markdown-parsed bodies and restored to LaTeX wrappers:

Theorem 1. *For every real number x, $x^2 \geq 0$.*

Proof. The square of a real number is nonnegative. □

TikZ settings are hoisted before the picture that uses them. Settings declared inside a tikzpicture are intentionally not globalized: \tikzset{regression node/.style={draw,rounded corners,inner sep=3pt}} \begin{tikzpicture} \node[regression node] {TikZ works}; \end{tikzpicture}

A complete LaTeX fence keeps its document body and rescues recognized preamble definitions. The duplicate siunitx package request is filtered out.

Recovered command: $\mathbb{R}$. Recovered optional argument: x^2 and x^3. Recovered document command: $\mathbf{x}$. Recovered declared command: x. Recovered math declarations: $\operatorname{rank}(A)$, $|x|$, and $\mathbb{N}$. Recovered Unicode declaration: **Unicode recovered from a LaTeX fence**.

Regression Claim 1. The regression fixture exercises the locally scoped theorem definition.

> The recovered custom environment also wraps its body.

LaTeX/TeX fences are rendered, not executed. The source of a LaTeX fence is hidden by default; add show=true when the source itself should appear in the PDF. Unrecognized or malformed preamble commands may not be rescued. The `tex` language alias is also accepted:

```tex
\[
\sum_{k=1}^{n} k = \frac{n(n+1)}{2}.
\]
```

$$\sum_{k=1}^{n} k = \frac{n(n+1)}{2}.$$

The fence parser also respects marker length. The shorter inner fence below must remain inside the four-backtick Markdown example rather than closing it:

```python
This is displayed Markdown code, not an executable Python block.
```

5. Python execution, presentation, and cache behavior

An omitted run attribute and run=true both mean “run normally”. A cold run asks you to approve the displayed source before it executes; an unchanged later build can use the cache without asking. Enter Y to approve. `trusted=true` skips the prompt only for ordinary uncached runs. run=new and cache=false always ask before executing.

```
print(”Python runs by default.”)
print(”Standard-library arithmetic:”, sum(range(1, 6)))
```

Python error - this block did not finish:

```
No installed Python found!
```

The following three examples contrast the execution modes with a simple plot. Install NumPy and Matplotlib to see both graphs (`py -m pip install numpy matplotlib`):

Ordinary Python (no run attribute) executes when needed. Its graph and output are reused on an unchanged cached build:

```
try:
    import numpy as np
    import matplotlib.pyplot as plt
    x = np.linspace(0, 10, 100)
    plt.figure()
    plt.plot(x, np.sin(x), label=”sin(x)”)
    plt.title(”Default Python: cached plot”)
    plt.legend()
    plt.tight_layout()
    plt.show()
    print(”DEFAULT_MODE plot generated.”)
except ImportError as exc:
    print(”SKIP plot: install numpy and matplotlib:”, exc)
```

Python error - this block did not finish:

```
No installed Python found!
```

`run=new` asks for confirmation and runs the simulation again on each build, then refreshes the cache with the new graph and output:

```
try:
    import numpy as np
    import matplotlib.pyplot as plt
    x = np.linspace(0, 10, 100)
    plt.figure()
    plt.plot(x, np.cos(x), color="orange", label="cos(x)")
    plt.title("run=new: freshly executed plot")
    plt.legend()
    plt.tight_layout()
    plt.show()
    print("RUN_NEW_MODE plot generated.")
except ImportError as exc:
    print("SKIP plot: install numpy and matplotlib:", exc)
```

`run=false` does not execute Python and produces no graph; it leaves the source visible as a code listing:

```
import matplotlib.pyplot as plt

plt.plot([1, 2, 3], [4, 5, 6])
plt.show()
```

The Python runner presents **file** in the input working directory and keeps that directory on sys.path:

Python error - this block did not finish:

```
No installed Python found!
```

The py, python3, and py3 class aliases are recognized:

Python error - this block did not finish:

```
No installed Python found!
```

Python error - this block did not finish:

```
No installed Python found!
```

Python error - this block did not finish:

```
No installed Python found!
```

run=true does not bypass the cache. run=new asks for confirmation and forces this block to execute again for this build, then refreshes the cache entry for a later ordinary build:

```
print("This run=new block is deliberately fresh on every build of this fixture.")
```

cache=false is the alternate cache-bypass option and also asks every time:

```
print("This cache=false block also executes on every build.")
```

show=false hides source; output=false hides standard output; maxlines limits displayed standard output:

Python error - this block did not finish:

```
No installed Python found!
```

Python error - this block did not finish:

```
No installed Python found!
```

The explicit deps attribute adds the named file's content to the cache key:

Python error - this block did not finish:

```
No installed Python found!
```

run=false does not execute this recognized Python fence; it remains a normal code listing. The print statement is source text, not expected output:

```
print("This must not execute because run=false is set.")
```

The next block is an intentional, controlled failure. The traceback should be reported to the console and shown as an error block; the overall document should still compile:

```
raise RuntimeError("Expected regression demonstration: visible Python error is non-fatal to the PDF build.")
```

error=false keeps the failure on the console but hides its error block:

```
raise RuntimeError("Expected regression demonstration: error=false hides this block.")
```

Matplotlib is optional. When installed, this block saves a figure and the runner captures it for the PDF. Without Matplotlib, it reports a skip and the rest of the document continues:

```
try:
    import matplotlib.pyplot as plt
except ImportError:
    print("SKIP: install matplotlib to exercise Python figure capture.")
else:
    plt.plot([0, 1, 2], [0, 1, 4], marker="o")
    plt.title("Optional Matplotlib figure capture")
    plt.xlabel("x")
    plt.ylabel("x squared")
    plt.show()
    print("Matplotlib figure captured.")
```

Python fences may read/write ordinary local files. This pair checks cache restoration of a workspace-local side effect between dependent blocks:

Python error - this block did not finish:

```
No installed Python found!
```

Python error - this block did not finish:

```
No installed Python found!
```

The final cleanup block runs every time, including on a later build after the first block's cache has restored the generated file:

```
from pathlib import Path

Path("FILTER_FEATURE_REGRESSION.generated.csv").unlink(missing_ok=True)
print("Removed the disposable side-effect fixture.")
```

The cache tracks many file opens and restores eligible workspace-local side effects on a valid cache hit. Use deps="relative-or-absolute-path" for dependencies that automatic tracking cannot identify reliably. The generated CSV is removed by the cleanup block. Cache entries are scoped to the first input's working directory so side effects from another input folder are never restored here. If a recorded side-effect directory has been removed, the block is rerun to recreate it.

6. Images and unsupported inputs

Remote images and unsupported extensions are dropped with a warning instead of being passed to XeLaTeX as broken image paths:

The filter supports existing local PNG, JPG/JPEG, and PDF images. Missing local images are dropped with a warning. A Python-generated figure above exercises image embedding without requiring a separate fixture asset.

Raw HTML nodes represented by Pandoc as raw blocks/inlines are removed. HTML divs may instead become structural Divs, whose text content is retained. Selected HTML leftovers in prose (br, sub, sup) are converted by text normalization. HTML inside code spans, indented code, fenced code, and supported verbatim/listing environments stays literal.

7. Known constraints and error handling

Document-level constraints and expected error handling behavior:

- Missing input files are rejected before Pandoc starts.
- Controlled Python exceptions display tracebacks without halting the overall PDF build.
- Remote images (e.g. SVG) and missing local images are dropped with console warnings.
- Missing packages or invalid XeLaTeX syntax abort gracefully without overwriting existing output.
- Bibliography support uses Pandoc citeproc conventions rather than raw natbib/biblatex.
- For active, executable failure tests (syntax errors, compilation failures, unclosed fences), see `compiled-fence-test.md`.

8. Validation checklist

After compiling this file:

- PDF exists beside this .md file.
- PDF title and author match the YAML metadata.
- With no `title-rule` option, the horizontal title rule remains visible (setting `title-rule: false` in YAML front matter disables it).
- Both citation text and a CSL-formatted reference list appear.
- Headings, lists, table, inline/display math, theorem, proof, and TikZ render.
- Real Python output appears; run=false source appears without its output.
- An uncached default Python block asks for approval; a valid cache hit does not. `trusted=true` skips only the normal first-run prompt.
- The ordinary probability graph is cached when unchanged; run=new generates its simulation again; run=false shows source without executing or drawing.

- run=new and cache=false prompt before execution on every build and show output after approval.
- The dependent Python blocks can restore/read their cached CSV on a later build.
- Cache results are scoped to this input's directory; missing side-effect directories invalidate an entry and rerun the block instead of failing.
- The controlled Python failure is visible on the console but does not stop PDF creation.
- Matplotlib figure appears when the optional dependency is installed.
- Console warnings explain intentionally removed markers/emoji and dropped remote SVG.
- The input .md file remains unchanged.

Expected output file: filter-feature-regression.pdf

Presentation page-size check

To check the optional page-size preset without changing this broad fixture, save the following as `page-layout-presentation.txt` and compile it through compile.bat:

```
Presentation page-size check.
```

The resulting PDF should report a page size of 960 x 540 points (16:9 landscape) with pdfinfo. A document without page-layout metadata remains A4. An unsupported page-layout value should print a warning and produce an A4 PDF.

Title-rule check

The title rule is visible by default. Set `title-rule: false` as a YAML boolean to omit it; the space below the title remains unchanged. Quoted `"false"` is a string, not a boolean, and warns while keeping the rule.

Multi-file regression

The single-input build above does not test multi-file merging. From the tool root, run this additional build with the companion input as the second file:

```
.\compile.bat `
    ".\documentation\filter-feature-regression.md" `
    ".\documentation\filter-feature-regression-part-2.md"
```

The resulting PDF is again written beside the first input. Extract or inspect its text and verify the four unique markers occur exactly once in this order:

1. The plain-text marker from input 1.
2. The Python output marker from input 1.
3. The plain-text marker from input 2.
4. The Python output marker from input 2.

This confirms the two input documents are combined in command-line order and the Python fence from each input contributes one output. It does not mean each fence is run once per input; each recognized fence is processed once in the combined document.

9. 2026-11-05 normalizer fix update

The table above also covers the nested-formatting fix. In the PDF, **Bold *emphasis before*** and **and after remains bold** should both be bold, with the matrix rendered as math between them: $\begin{matrix} a & b \\ c & d \end{matrix}$. The matrix is outside the formatting spans. Before the fix, nested formatting around complex table math could cause a XeLaTeX error such as "Forbidden control sequence … `\check@nocorr@`" or lose surrounding text.

To check mixed-input fallback, run this fixture with compile.bat as the second input from the tool root:

```
.\compile.bat `
    ".\documentation\filter-feature-regression.md" `
    ".\compile.bat"
```

The unsupported .bat input intentionally makes the normalizer retain Pandoc's original parse instead of using its cleaned-input path. The PDF will include compile.bat as input content; that is expected for this test.

Before the fix, cleanup state gathered from this fixture could leak into finalization even though the original parse was retained. The console could report removed citation-marker characters while the original citation markers remained in the PDF.

After the fix, the following markers should remain visible in the PDF and the console should not report citation-marker removals from the abandoned cleanup path:

Fallback marker one: FALLBACK_STATE_ALPHA

Fallback marker two: FALLBACK_STATE_BETA

References

[] Doe, Jane. 2024. “A Small Reference for Workflow Testing.” *Regression Notes*.